主管面（二面）

非技术面试准备材料 · 答题模板

1. 请先做一个自我介绍

我有【X】年开发经验，最近两年主要做 AI 应用和 Agent 方向。最能代表我当前能力的是 AI4Test 项目：目标是把软件测试的全流程做成一套多 Agent 平台，覆盖需求分析、测试设计、用例生成、环境准备、执行和结果分析。

我在项目里不是只负责某一个模块，而是从整体方案、核心链路到业务落地都参与了，包括 Agent 运行时、动态 Workflow 编排、工具调用与失败处理，以及效果评测和反馈机制。推进过程中需要和测试同事紧密协作，把他们的经验整理成系统能执行的规则和流程。

我觉得自己和这个岗位比较匹配的有三点：一是我既做过底层工程，也做过业务落地，知道怎么在可靠性和迭代速度之间做取舍；二是我习惯先定义验收标准再动手，不追求 demo 效果；三是我愿意主动对齐和推动事情，而不是只守着自己负责的那一块。

2. 详细介绍一个你最有代表性的项目

我讲一下 AI4Test 这个项目，按背景、我的职责、做法和结果来说。

背景是测试团队的工作链条很长，从需求理解、测试点设计、用例编写，到环境准备、执行和失败分析，很多环节依赖个人经验，重复劳动多，而且不同人做出来的质量差异比较大。我们希望用 Agent 把其中可标准化的部分承接起来，让人聚焦在判断和决策上。

我在项目里主要负责整体方案设计和核心链路落地：一是 Agent 运行时，处理一次任务的状态、上下文、工具调用和中断恢复；二是多 Agent 的编排，根据任务选择合适的 Agent 并组织执行顺序；三是工具执行和失败处理，保证有副作用的操作不会因为重跑而重复发生；四是评测和反馈机制，让效果可以被衡量和持续迭代。

做法上我没有一上来就追求全流程自动化，而是先选一个典型场景做端到端闭环：把需求、测试点、步骤、预期结果这条链路跑通，让测试同事参与 review，把他们的修改意见沉淀成规则和评测用例。等这个场景稳定后，再逐步扩展到更多阶段。

结果是这个 Agent 从最初"看起来能用但不敢用"，变成了测试同事愿意在日常工作里实际使用的工具，人工修改率逐步下降，一些重复性的用例整理工作明显减少。更重要的是我们形成了一套可以复用的协作和评测机制，后面扩展新场景时不用再从零开始。

3. 这个项目的整体架构是怎么设计的？为什么这么分层？

我整体是按"能力分层、编排在上"来设计的。

最底层是模型接入层，把不同厂商 API 的差异收敛掉，只回答"这个请求在某个厂商上应该怎么写"；往上是模型网关，负责"用哪个模型、挂了怎么切、成本和限流怎么控制"。再往上是 Agent 运行时，管理一次任务的状态、上下文、工具调用和中断恢复。最上面是多 Agent 编排层，根据任务目标选择 Agent、生成执行图并调度。

这么分主要有三个原因。第一是解耦，模型和业务分开，换模型或加厂商不用改业务代码。第二是故障边界清晰，重试和降

级只在网关这一层做，避免上下层重复重试导致风暴。第三是编排层可以独立演进，先支持固定流程，再支持模板化裁剪，最后才在必要的时候做动态生成。

分层也有代价，链路变长之后排查会更难，所以我在关键节点都保留了执行轨迹和结构化日志，任何一个结果都能沿着链路往回追。另外我们用了依赖方向的静态检查，防止底层模块反向依赖上层业务，保证架构不会随着迭代慢慢腐化。

4. 项目里最关键的取舍是什么？

最关键的取舍是"动态能力"和"确定性"之间怎么平衡。

Agent 系统很容易走向两个极端：一种是全部写死成固定流程，稳定但不够灵活；另一种是让模型自由规划，灵活但结果不可控，出了问题也很难复现。我最后采用的是分层的策略：生产环境优先走预定义或模板化的 Workflow，只有在需求变化大、没有合适模板时才启用动态生成，而且动态生成的结果要经过依赖和输入输出校验才能执行。

第二个取舍是效果和成本、时延之间。不是所有任务都值得用最强的模型和最复杂的验证，我会按任务风险分级：简单任务用轻量策略，高风险任务才启用多重校验和人工确认。

第三个取舍是自研和复用。现成框架能省时间，但可观测性和失败处理往往不够贴合我们的场景。我的原则是核心链路自己掌控，外围能力尽量复用，不为了"造轮子"而造轮子。

这些取舍背后其实是一个判断：工业场景里，可解释、可回滚、可验证比"看起来很智能"更重要。

5. 你怎么衡量这个项目做得好不好？

我会分业务指标和技术指标两层来看，而且一定要有基线。

业务侧主要看四件事：有多少测试同事真的在用、每周使用频次；单个任务相比人工节省了多少时间；Agent 产出被人工修改的比例；以及有没有帮助更早发现遗漏的测试点或缺陷。最核心的是使用率和人工修改率，因为前者说明有没有进入工作流，后者说明产出质量到底能不能用。

技术侧看任务成功率、步骤可执行率、关键测试点覆盖率、平均时延和单任务成本，以及故障恢复时间。这些指标主要用于定位问题，而不是拿来做汇报。

做法上我会先整理一套带标注的评测集，把历史真实需求和缺陷作为回归样本，每次改动都跑一遍，避免"改好一个场景、弄坏另一个场景"。上线后通过灰度和使用数据继续验证。

我不会只看模型回答得像不像，也不会只看 demo 效果。AI 应用最容易犯的错误是拿主观感受当指标，所以能自动化的指标尽量自动化，不能自动化的就请业务同事按清单评审。

6. 需求很多、资源有限时，你怎么排优先级？

我一般按三步来排。

第一，先对齐目标和约束。我会和业务方、主管确认这个阶段最重要的目标是什么，是验证可行性、提升效率，还是保证稳定性。目标不同，优先级完全不一样。

第二，用"价值、紧迫性、成本与风险"一起判断，而不是只看谁催得急。我会把需求分成必须做、应该做、可以做三类，优

先保证能验证核心假设的最小闭环。比如项目初期我没有同时铺开所有测试阶段，而是先把一个场景端到端跑通，因为它能最早暴露业务定义和工具链的问题。

第三，明确"不做什么"并同步出去。资源有限时，最怕的不是少做，而是所有人都以为自己的需求在做。我会把排期和取舍原因写清楚，定期同步进展和风险，需要升级的时候及时升级，而不是到最后才说做不完。

如果两个需求价值接近，我会优先做那个能沉淀通用能力、后续复用成本更低的。

7. 项目进行中需求突然变更，或者被紧急插队，你怎么处理?

我不会直接拒绝，也不会默默加班硬扛，而是先把变更的影响算清楚。

第一步判断变更性质：是目标变了，还是实现方式变了。如果目标变了，那排期和范围本来就应该重新讨论；如果只是实现方式调整，我会评估对当前工作的影响。

第二步给出可选方案，而不是把问题原样丢回去。通常有三个选项：保时间就砍范围、保范围就加资源或延期、或者先上一个临时方案。我会说明每个选项的代价和风险，让决策者做选择。

第三步执行并留痕。确定之后更新排期和范围，把变更原因和影响记录清楚，避免后面反复扯皮。

如果是线上问题这类真正的紧急事项，我会先止损、再排修复，同时评估对当前迭代的影响并同步。我的原则是：变更可以接受，但必须是有意识的取舍，而不是悄悄地让团队承担所有成本。

8. 项目里最难的技术问题是什么？你是怎么解决的?

最难的不是模型效果，而是"跑到一半"这个状态。

Agent 的一次任务可能执行几十步、调用很多次工具。如果进程在中途被杀、模型超时或者上游限流，恢复的时候就会遇到一个很麻烦的情况：某个工具有副作用，可能已经执行了，但结果还没回填。如果直接按检查点重跑，就可能重复写文件、重复发请求；如果不重跑，又可能漏掉结果。

我的处理分三层。第一，在工具层把"执行中"和"结果不确定"显式建模，超过租约还没有回执的调用标记为不确定，而不是当成失败。第二，在运行时层恢复时先核对工具状态，再决定是重放、跳过还是转人工确认，而不是只看检查点里有没有快照。第三，对关键操作做幂等设计，尽量让重试是安全的。

这件事让我印象很深，因为它不是靠调模型能解决的，而是要把分布式系统里"状态不确定"的思路搬到 Agent 里。解决之后，续跑时的重复副作用基本消失了，也让我们敢把更长的任务交给 Agent。

9. 线上出过什么问题？你是怎么处理的?

比较典型的一次是模型侧不稳定导致的连锁失败。上游限流之后，任务开始超时，重试又把压力放大，最后一批任务堆积，业务同事看到的现象是"Agent 卡住了"。

处理上我分四步。第一是止损，先限制并发、关闭自动重试，把堆积的任务排队或转人工，避免影响继续扩大。第二是定位，通过执行轨迹和网关指标确认问题出在模型侧，而不是我们的工具或业务逻辑。第三是修复，调整重试策略和超时预算，加上熔断和降级模型，让单点故障不会拖垮整个链路。第四是复盘，把这次问题写成记录，明确哪些是机制缺失、哪些是监控不够，并补上告警和演练。

我处理线上问题的原则是：先恢复，再解释；先看证据，再下结论；复盘对事不对人，重点看机制为什么没拦住。修复完之后我会补一个回归用例，确保同类问题能被提前发现。

10. 你怎么保证交付质量？

我会把质量拆到几个环节，而不是最后靠一次测试兜底。

开发阶段做分层测试：核心逻辑有单元测试，模块之间有契约测试，关键链路有集成测试。模型相关的部分用确定性的 `mock` 跑 `CI`，避免因为模型波动导致测试不稳定。

上线前做评测和灰度。我们维护了一套带业务标注的评测集，覆盖典型场景和历史失败案例，每次改动都跑回归；新能力先小范围灰度，观察成功率、人工修改率和成本，再逐步放量。

上线后做可观测和回滚。关键节点有执行轨迹、工具调用记录和结构化日志，出问题能快速定位；版本和配置都可以回滚，避免一次发布造成不可逆影响。

另外我会把线上发现的问题补进评测集，形成闭环。对 `AI` 应用来说，评测集其实就是我们的回归测试，没有它，质量只能靠感觉。

11. 在团队协作中你通常是什么角色？怎么推动不向你汇报的人？

我通常是比较偏"推进和落地"的角色：把模糊的目标拆成可执行的任务，把跨角色的信息对齐，然后盯住关键路径往前走。

推动不归我管的人，我主要靠三点。第一是找共同目标，先讲清楚这件事对他有什么价值，而不是只讲我的需求。第二是降低对方的参与成本，比如先把方案、接口和数据准备好，让对方只需要做判断，而不是从零配合。第三是用小范围试点和结果说话，先做出一个能看得见效果的样例，再推广，比开会说服更有效。

过程中我会明确接口人、交付物和时间点，重要结论落到文档里。遇到推不动的情况，我会先判断是优先级问题、资源问题还是信息不对称，再决定是调整方案还是向上同步风险。我不会用"告状"的方式推动事情，但也不会让风险一直藏着。

12. 你有带人或者做知识沉淀的经验吗？

我没有正式带过团队，但在项目里承担过类似的部分。

一是带过新同事熟悉项目，我会先给他一张系统全景图，再安排一个边界清晰、能独立交付的小任务，过程中通过 `code review` 和定期同步帮他建立判断，而不是直接把活丢过去。

二是做过方案评审和技术分享，把 `Agent` 运行时、工具失败处理、评测方法这些内容整理成文档和分享材料，让团队里其他人能复用，而不是只存在我脑子里。

三是推动了一些规范，比如依赖方向检查、配置和版本管理、问题复盘模板。这些事不显眼，但能减少后面反复踩坑。

如果后续有机会带人，我希望自己先做到目标清楚、边界清楚、反馈及时，而不是靠盯人来保证结果。

13. 如果你和直属领导意见不一致，你会怎么做？

我会先假设自己可能信息不全。领导通常能看到更多业务优先级、资源约束或者上层目标，所以我会先问清楚背景，而不是急着反驳。

了解之后如果仍然不认同，我会私下沟通，重点讲风险和替代方案，而不是只说"我觉得不行"。能验证的分歧尽量用数据或小范围试点来验证，把主观争论变成事实判断。

如果最后决策没有改变，我会按决定执行，同时把自己担心的风险点和兜底方案记录下来，提前做好预案。我认为职业性体现在：会上可以充分讨论，会后不消极执行，也不阳奉阴违。

当然，如果是涉及数据安全、合规或者明显会伤害用户的事情，我会明确表达并坚持，这种情况不能只用"服从"来处理。

14. 你希望加入什么样的团队？喜欢什么样的管理风格？

我希望加入一个目标清晰、重视工程质量的团队。

具体来说，一是方向要清楚，知道当前最重要的问题是什么，而不是同时铺很多事；二是愿意为长期能力投入，比如评测、可观测性、文档和规范，而不是只追求短期 `demo`；三是沟通直接，有问题当面说，不用猜。

管理风格上，我比较适应"目标清楚 + 充分授权 + 及时反馈"的方式。给定目标和边界后，我希望能自己安排节奏，关键节点对齐，遇到风险及时升级。我不太需要手把手的微观管理，但希望反馈是具体和及时的，而不是到年底才说哪里不行。

同时我也会主动承担，不会等着被安排。团队需要有人做那些不显眼但重要的事时，我愿意去做。

15. 如果入职，你未来一年打算怎么开展工作？

我会按"先融入、再交付、后优化"的节奏来。

前三个月，重点是理解业务和系统：业务目标、核心流程、系统边界、现有技术栈和协作方式。同时争取独立交付一个完整的模块或需求，从方案到上线走一遍，验证自己的理解，也建立和上下游的信任。

三到六个月，希望在负责的范围内做出可衡量的结果，比如提升某个环节的效率或稳定性。这个阶段我会重点补评测和可观测性，让效果不是靠感觉判断。

六到十二个月，希望能从一个点扩展到一个方向，沉淀出可复用的方法和工具，并带动团队一起用。如果有机会，我也愿意承担方案设计、评审和带新人的工作。

整体上我不会一上来就提大改造，而是先理解为什么现在是这么做，再判断哪些值得改、哪些其实是合理的历史选择。

16. 你还有什么问题想问我们？

我有三个问题。

第一，这个岗位未来半年最重要的目标是什么？团队现在最希望解决的问题是哪一类？

> 第二，团队目前的技术栈和协作方式是怎样的？这个岗位会更多独立负责，还是和业务、算法、工程同学紧密配合？
>
> 第三，您理想中这个岗位做得好的人，应该具备哪些特质？我也想对照一下自己的差距。
>
> 另外如果方便的话，我也想了解一下团队对这个方向未来一两年的判断。

使用说明

- 文中【X】【期望区间】等占位符，请按自己的真实经历和数据替换。
- 第 2、8、9 题建议准备成可复述的完整故事，面试时按"背景—职责—做法—结果—反思"展开，控制在 2 到 3 分钟。
- 涉及指标的地方，优先说真实数据；如果没有精确数据，用"明显下降""从不可用到可用"这类可解释的表述，不要编造数字。